

Privacy-Preserving Anomaly Detection for Industrial Control Systems using Homomorphic Encryption

M.Tech Thesis Project — Indian Institute of Technology Ropar (2024–2026)

Can we detect cyberattacks on industrial systems without ever decrypting the sensor data?

This project answers yes — using Fully Homomorphic Encryption.

Table of Contents

- [Overview](#)
 - [Motivation](#)
 - [System Architecture](#)
 - [Methodology](#)
 - [Plaintext Baselines](#)
 - [HE Protocol Design](#)
 - [Multi-Round Interactive Protocol](#)
 - [Results](#)
 - [Project Structure](#)
 - [Setup & Installation](#)
 - [How to Run](#)
 - [Dataset](#)
 - [Future Work](#)
 - [Citation](#)
 - [Author](#)
-

Overview

This project implements a **privacy-preserving anomaly detection system** for Industrial Control Systems (ICS/OT) using **Fully Homomorphic Encryption (FHE)**. The core idea is that a server can perform anomaly detection on encrypted sensor data — without ever seeing the raw values.

Two anomaly detectors are implemented and evaluated under encryption:

- **HE-SAD** — Homomorphic Encryption based Statistical Anomaly Detection
- **HE-PCA** — Homomorphic Encryption based Principal Component Analysis

Both are evaluated on the **SWaT (Secure Water Treatment)** benchmark dataset — a widely used real-world ICS security dataset containing actual cyberattack scenarios.

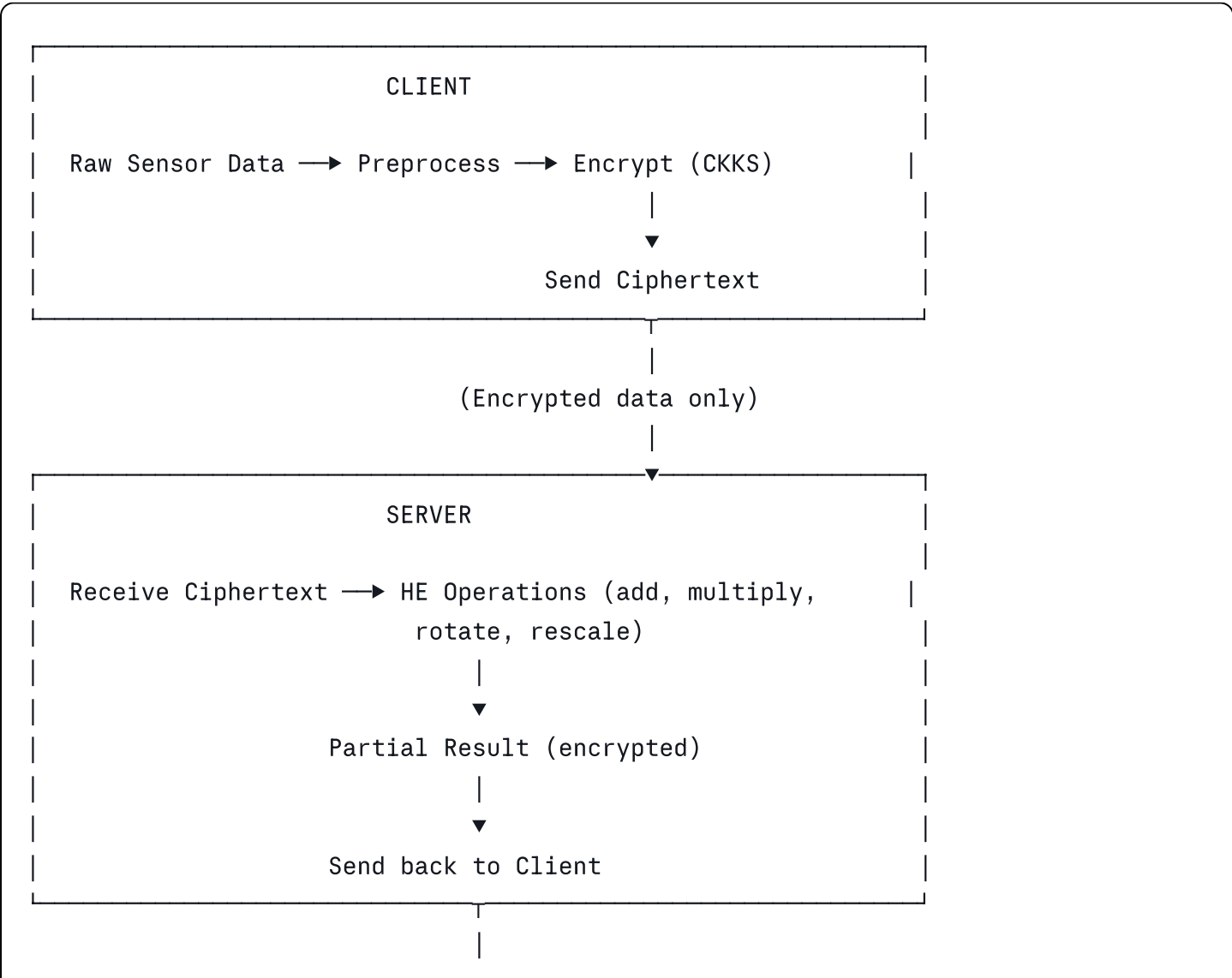
Motivation

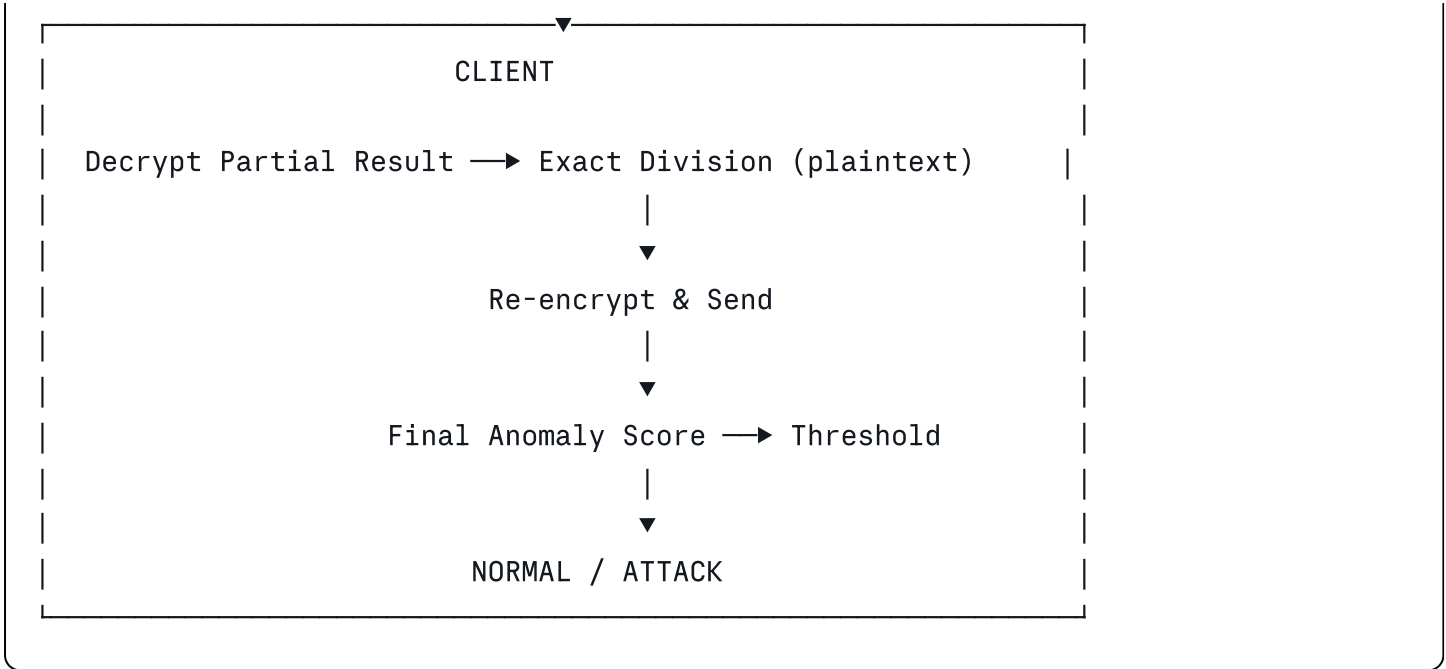
Industrial Control Systems (ICS) power critical infrastructure — water treatment plants, power grids, manufacturing lines. Detecting anomalies (cyberattacks, sensor faults) in these systems is critical for safety.

However, sharing raw sensor data with an external monitoring service raises serious **privacy and sovereignty concerns** — especially in regulated industries. Homomorphic Encryption solves this: it allows a third-party server to run detection algorithms on **encrypted data**, returning encrypted results that only the data owner can decrypt.

The challenge: Standard anomaly detection algorithms rely on operations like division, square roots, and statistical comparisons — which are expensive or inaccurate inside FHE. This work addresses that challenge head-on.

System Architecture





The key innovation is the **multi-round interactive protocol** — instead of approximating division inside FHE (which introduces significant error), the client handles exact division in plaintext between rounds. This preserves numerical precision while keeping raw data private throughout.

Methodology

Plaintext Baselines

Before implementing HE versions, plaintext baselines were established:

Detector	Description
SAD	Statistical Anomaly Detection using mean and standard deviation per feature; flags samples exceeding a threshold number of standard deviations
PCA	Principal Component Analysis reconstruction error; anomalies produce high reconstruction loss

HE Protocol Design

The encryption layer uses the **CKKS scheme** from Microsoft SEAL — chosen for its support of approximate arithmetic on real-valued data, which suits continuous sensor readings.

SEAL Parameters:

Parameter	Value
Scheme	CKKS
Polynomial Modulus Degree	8192
Coefficient Modulus	[60, 40, 40, 60]
Scale	2^40

Attempted approaches (and why they were abandoned):

- **Chebyshev Polynomial Approximation** of division — high approximation error for small denominators common in ICS data
- **Newton-Raphson Reciprocal Approximation** — converges slowly inside HE due to multiplicative depth limits; significant AUC degradation observed

Both are documented in the codebase as research artifacts under `experiments/`.

Multi-Round Interactive Protocol

The final protocol avoids approximation entirely:

1. **Round 1:** Client encrypts preprocessed data → sends to server
2. **Server:** Computes encrypted numerator and denominator separately using HE additions and multiplications
3. **Round 2:** Server sends encrypted partial results back to client
4. **Client:** Decrypts, performs **exact division in plaintext**, re-encrypts the result
5. **Round 3:** Client sends re-encrypted result to server for final thresholding
6. **Server:** Returns encrypted anomaly flag → client decrypts → decision

This design eliminates the root cause of HE division error while maintaining the privacy guarantee — the server never sees raw values at any round.

Results

SWaT Dataset

Method	AUC	F1 Score	Notes
Plaintext SAD	~0.94	0.760	Baseline
Plaintext PCA	~0.93	0.725	Baseline

Method	AUC	F1 Score	Notes
HE-SAD (Multi-Round)	0.9533	—	Exceeds plaintext baseline
HE-PCA	0.9344	—	Matches plaintext closely
HE-SAD (Newton-Raphson)	~0.71	—	Approximation error degrades AUC
HE-SAD (Chebyshev)	~0.68	—	High error for small denominators

✅ The multi-round interactive protocol achieves **HE-SAD AUC = 0.9533**, exceeding the plaintext SAD baseline — demonstrating that privacy-preserving detection need not compromise accuracy.

Project Structure

```
.
├── system/                # Shared utilities and configuration
│   ├── config.py          # SEAL parameters, thresholds, paths
│   ├── preprocessing.py   # SWaT dataset loading, normalization
│   └── utils.py           # Helper functions, metrics (AUC, F1)
│
├── client/               # Client-side encryption and decryption
│   ├── keygen.py          # Key generation (public, secret, relin, galois)
│   ├── encrypt.py         # Data encryption using CKKS
│   ├── decrypt.py         # Decryption and re-encryption for multi-round
│   └── client_pipeline.py # Full client-side orchestration
│
├── server/              # Server-side HE computation
│   ├── he_sad.py          # HE-SAD: encrypted mean/std computation
│   ├── he_pca.py          # HE-PCA: encrypted reconstruction error
│   └── server_pipeline.py # Full server-side orchestration
│
├── experiments/         # Research artifacts (approximation attempts)
│   ├── chebyshev_approx.py # Chebyshev division approximation (archived)
│   └── newton_raphson.py   # Newton-Raphson reciprocal (archived)
│
├── baselines/           # Plaintext baseline implementations
│   ├── sad_baseline.py    # Plaintext SAD
│   └── pca_baseline.py    # Plaintext PCA
│
├── data/                # Dataset directory (not tracked by git)
│   └── SWaT_Dataset_Attack_v0.xlsx
│
└── results/             # Output logs, AUC curves, metrics
```

```
└─ requirements.txt
└─ README.md
```

Setup & Installation

Prerequisites

- Python 3.8+
- Microsoft SEAL Python bindings (`seal-python`)
- ~4GB RAM recommended (CKKS with degree 8192)

1. Clone the Repository

```
bash

git clone https://github.com/bhushandatre/fhe-anomaly-detection-ics.git
cd fhe-anomaly-detection-ics
```

2. Create a Virtual Environment

```
bash

python -m venv venv
source venv/bin/activate      # Linux/Mac
venv\Scripts\activate        # Windows
```

3. Install Dependencies

```
bash

pip install -r requirements.txt
```

requirements.txt includes:

```
numpy
pandas
scikit-learn
matplotlib
seal-python
openpyxl
tqdm
```

Note on SEAL installation: `seal-python` requires compilation from source on some systems. If `pip install seal-python` fails, follow the [official build guide](#).

4. Download the SWaT Dataset

The SWaT dataset requires a formal request to iTrust, Singapore University of Technology and Design: 🖱️ https://itrust.sutd.edu.sg/itrust-labs_datasets/dataset_info/

Place the downloaded file at:

```
data/SWaT_Dataset_Attack_v0.xlsx
```

How to Run

Run Plaintext Baselines

```
bash

python baselines/sad_baseline.py
python baselines/pca_baseline.py
```

Run Full HE Pipeline (Multi-Round Protocol)

```
bash

# Step 1: Generate keys and preprocess data
python client/keygen.py
python system/preprocessing.py

# Step 2: Run the full interactive pipeline
python client/client_pipeline.py
```

This will:

1. Encrypt the test data on the client
2. Send to server for HE computation
3. Return partial results for client-side division
4. Re-encrypt and complete the anomaly scoring
5. Output AUC, detection results, and save plots to `results/`

Run Individual Components

```
bash
```

```
# Server-side HE-SAD only
python server/he_sad.py

# Server-side HE-PCA only
python server/he_pca.py
```

Run Approximation Experiments (archived)

```
bash

python experiments/chebyshev_approx.py
python experiments/newton_raphson.py
```

Dataset

SWaT (Secure Water Treatment) is a real-world ICS dataset from a water treatment testbed at SUTD, Singapore. It contains:

- 11 days of continuous sensor/actuator readings (946,722 samples)
- 7 days normal operation + 4 days with 41 staged cyberattacks
- 51 features (sensors and actuators)
- Binary labels: Normal / Attack

 Due to data use agreements, the SWaT dataset is **not included** in this repository. Request access directly from iTrust SUTD.

Future Work

- ☐ **BATADAL dataset evaluation** — extend HE pipeline to the BATADAL water distribution dataset
 - ☐ **Communication overhead reduction** — compress ciphertexts between client-server rounds
 - ☐ **Multi-party computation (MPC) hybrid** — combine HE with secret sharing for distributed ICS monitoring
 - ☐ **Deep learning under HE** — explore CKKS-compatible encrypted LSTM/CNN for time-series anomaly detection
 - ☐ **Real-time streaming pipeline** — adapt batch pipeline to sliding window for live ICS monitoring
 - ☐ **Conference paper submission** — formalise results for IEEE S&P / USENIX Security / CCS
-

Citation

If you use this work or find it useful, please cite:

bibtex

```
@mastersthesis{datre2026fhe,  
  author      = {Bhushan Datre},  
  title       = {Privacy-Preserving Anomaly Detection for Industrial Control Systems u  
  school      = {Indian Institute of Technology Ropar},  
  year        = {2026},  
  type        = {M.Tech Thesis}  
}
```

Author

Bhushan Datre

M.Tech CSE — Indian Institute of Technology Ropar (2024–2026)

 2024CSM1004@iitrpr.ac.in

 github.com/bhushandatre

 linkedin.com/in/bhushan-datre-a8597621b

"Privacy is not about hiding something. It is about protecting something."

This project is a step toward making critical infrastructure monitoring both secure and private.